Estructura de Datos y Algoritmia
Tema 3: Estructuras de datos lineales

Carlos Pérez

Departamento de Lenguajes y Sistemas Informáticos
Universidad de Alicante
Curso 2026-2027



Esta obra se publica con una licencia BY-NC-SA Creative Commons License

Index

1. Objetivos
2. Librerías de Java y C++
3. Vectores
4. Listas
5. Pilas
6. Colas

Index

1. Objetivos
2. Librerías de Java y C++
3. Vectores
4. Listas
5. Pilas
6. Colas

1. Objetivos

- describir estructuras de **datos lineales**, sus operaciones y tiempos de ejecución;
- implementar y usar las estructuras de datos lineales;
- conocer las estructuras de datos lineales que hay en las librerías de `java.util` del lenguaje de programación Java;
- conocer las estructuras de datos lineales que hay en las librerías `STL` del lenguaje de programación C++;
- conocer algunas aplicaciones de estructuras de datos lineales.

Index

1. Objetivos
2. Librerías de Java y C++
3. Vectores
4. Listas
5. Pilas
6. Colas

Java Collections Framework

La plataforma Java incluye un API (en el paquete `java.util`) para manipular **colecciones de elementos** independientemente de su implementación:

- interfaces y clases (implementaciones concretas)
- iteradores, que permiten hacer recorridos
- algoritmos
- manipulación de arrays

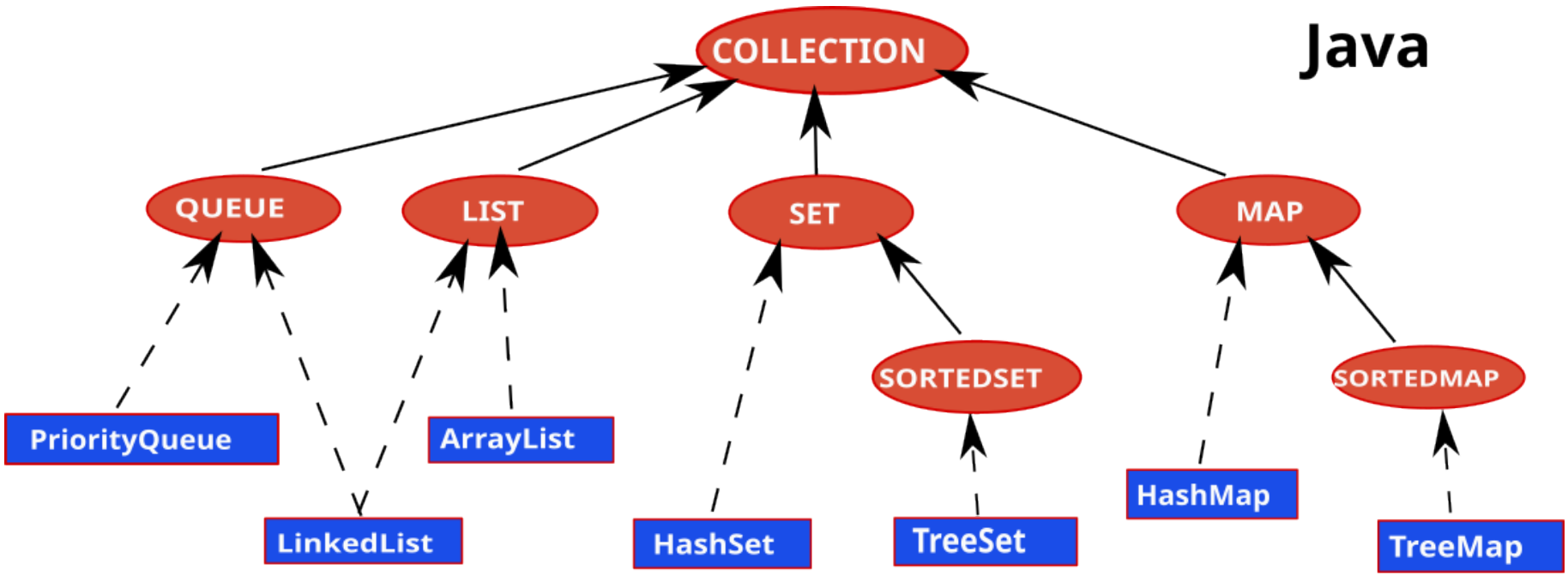
En Java, se emplea la interfaz genérica `Collection`, que permite almacenar cualquier tipo de elemento y usar una serie de métodos comunes: añadir, eliminar, obtener el tamaño de la colección, etc ...

Partiendo de `Collection` se extiende otra serie de interfaces especializadas.

Para incluir las librerías usaremos `import java.util.*`

`https://docs.oracle.com/javase/8/docs/api/java/util/package-summary.html`

Interfaces y clases en Java



Biblioteca STL (Standard Template Library)

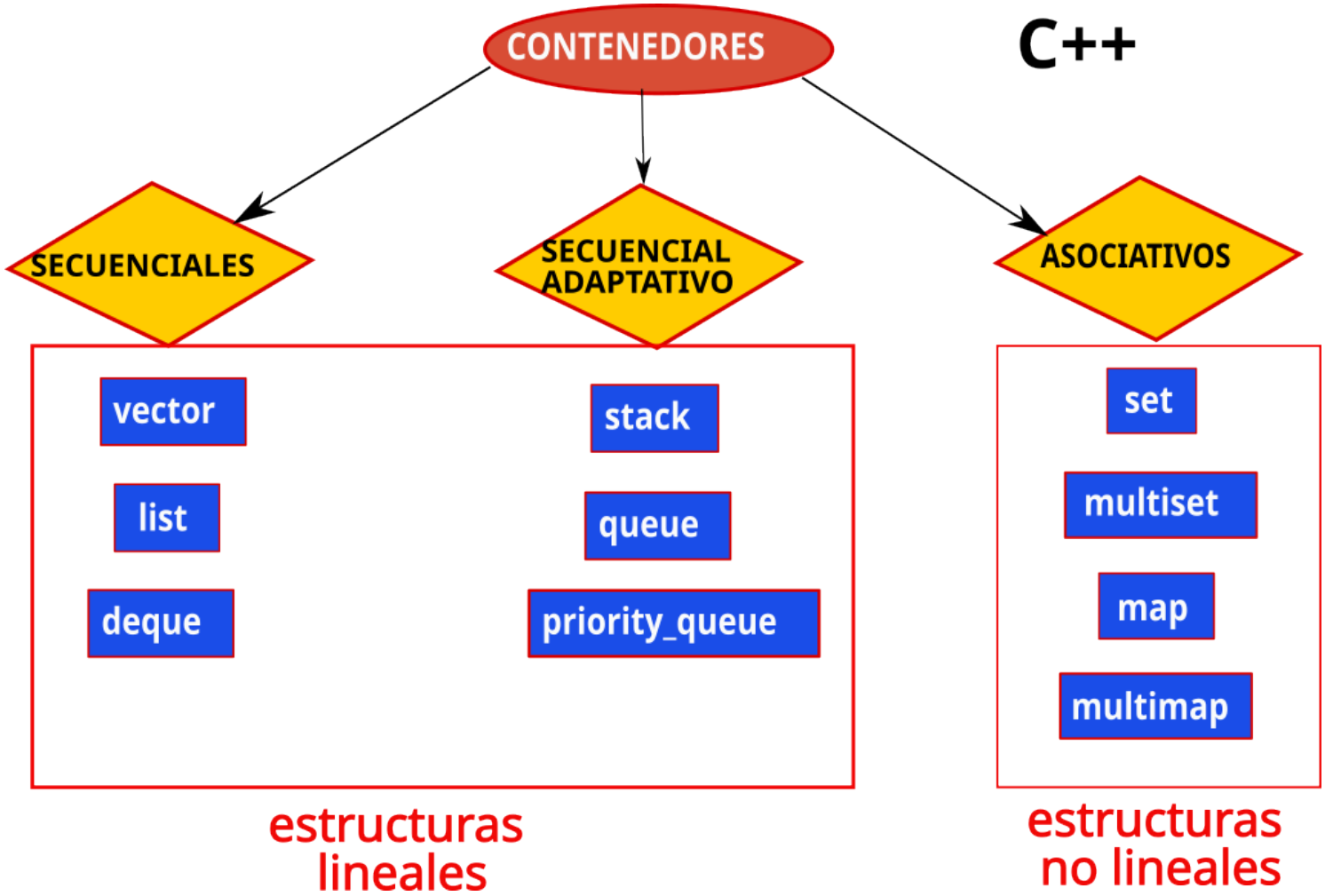
La biblioteca de plantillas estándar (STL) es la biblioteca estándar de C++ que proporciona programación genérica para muchas estructuras de datos y algoritmos:

- clases contenedoras (vector, list ...)
- iteradores, que permiten hacer recorridos
- algoritmos

Para acceder a la biblioteca STL hay que hacer un `#include` para incluir uno o más ficheros de cabecera.

```
http://www.cplusplus.com/reference/stl/
```

Clases en C++



Adaptativo: permite cambiar un contenedor en otro modificando la interface del primero (p.ej. vector versus stack). Normalmente el nuevo sólo requerirá un subconjunto de las capacidades del original.

Ventajas de las librerías

- Estándar implica que está disponible por todos los compiladores y plataformas.
- Incrementa la productividad (los programadores no tienen que implementarla) y son libres de errores.
- Implementación rápida de aplicaciones ya que solo hay que seleccionar el algoritmo más rápido.
- Aumento de la legibilidad del código.
- Gestión de memoria propia (almacenamiento de memoria automático).

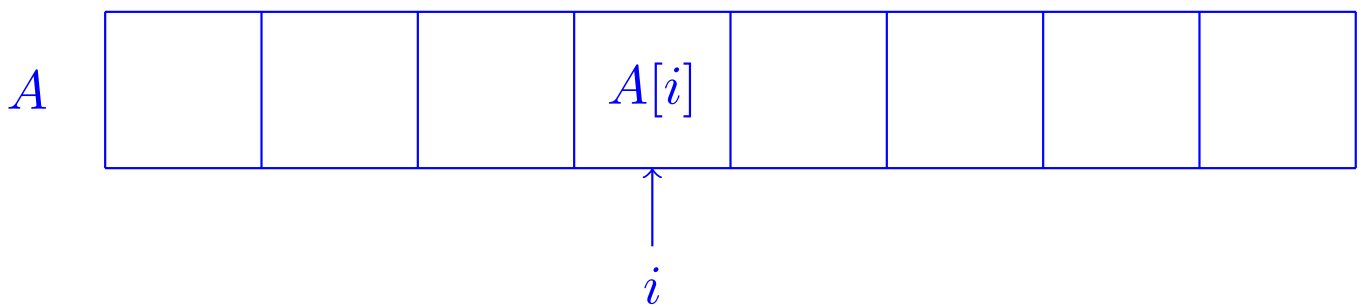
Index

1. Objetivos
2. Librerías de Java y C++
3. Vectores
4. Listas
5. Pilas
6. Colas

Concepto de vector

Los vectores se caracterizan porque:

- almacenan datos del mismo tipo
- cada dato almacenado tiene una posición (índice) asociada
- el índice será siempre un número entero mayor o igual a cero
- el **acceso** se pueda hacer en **tiempo constante**



java	**C++**
`int [] v1=new int[10];` `int v1[]={2,4,6};`	`int v1[10];` `int v1[]={2,4,6};`

Arrays en `java.util`

La clase `java.util.Arrays` contiene una batería de **métodos estáticos** útiles para la manipulación de arrays. Ejemplos con `int`:

Prototipo	Descripción
`static int binarySearch(int[] v, int c);`	busca c en v
`static int[] copyOf(int[] v, int w);`	copia w elementos en v
`static boolean equals(int[] v1, int[] v2);`	compara v1 y v2
`static void sort(int[] v);`	ordena v
`static String toString(int[] v);`	convierte v en String

Cuidado con cierto tipo de operaciones ...

```
import java.util.Arrays;
...

int v1[] = {8, 1, 2};
int [] v2 = new int[3];
int [] v3 = new int[3];

v2 = v1;
v3 = Arrays.copyOf(v1, 3);
Arrays.equals(v1,v2); //true o false?
Arrays.equals(v1,v3); //true o false?
Arrays.sort(v1);
Arrays.equals(v1,v2); //true o false?
Arrays.equals(v1,v3); //true o false?
```

¿Qué pasaría en C++?

Interfaces Collection y List

Java

Collection: interfaz "raíz" de todas las interfaces y clases

- `int size()`
- `boolean isEmpty()`
- `boolean contains(Object elem)`
- `Iterator iterator()`
- `Object[] toArray()`
- `boolean add(E elem)`
- `boolean remove(Object elem)`
- `void clear()`

List: extiende la interface `Collection`, especificando operaciones de una colección cuyos **elementos permanecen en un orden particular**.

- `void add(int index, E elem)`
- `E remove(int index)`
- `E get(int index)`
- `E set(int index, E elem)`
- `int indexOf(Object o)`

java.util.ArrayList

- está declarada como `public class ArrayList<E>`
- el almacenamiento se gestiona automáticamente (**array flexible**);
- acceso con índice entero (empezando en 0);
- coste constante: size, isEmpty, add (al final), get, set;
- coste lineal: búsqueda/borrado (por elemento), inserción/borrado por índice;

Prototipo	**Descripción**
`int size()`	dev número de elementos almacenados
`boolean isEmpty()`	dev `true` si está vacío
`boolean add(E o)`[1]	añade `o` al final del vector
`E get(int i)`	dev elemento en la posición `i`
`E set(int i, E o)`	dev elemento en `i` y asigna `o` al índice `i`
`boolean contains(Object o)`	dev `true` si contiene `o`
`boolean equals(Object o)`	dev `true` si `this` y `o` son iguales
`boolean remove(Object o)`	borra el primer elemento igual que `o`
`E remove(int i)`	borra el elemento en la posición `i` y desplaza el resto
`void add(int index, E o)`	inserta `o` en `i` y desplaza el resto

[1] amortizado

Uso de java.util.ArrayList

Java

Al principio del programa se añadirá `import java.util.Arraylist`

```
ArrayList<Integer> v1=new ArrayList<Integer>();

v1.add(1); //[1]
v1.add(5); //[1, 5]
v1.add(2); //[1, 5, 2]

int i = v1.get(0); //1
int j = v1.set(2,5); //2 y v1=[1, 5, 5]

v1.add(0, 7);//[7, 1, 5, 5]

//EL ARGUMENTO ES UN int Y SE ASOCIA A UN INDICE
v1.remove(5); //Exception in thread "main"
              //java.lang.IndexOutOfBoundsException: Index: 5, Size: 4
              //at ArrayListClase.main(ArrayListClase.java:20)

//EL ARGUMENTO ES UN Integer Y SE ASOCIA A UN Object DE TIPO Integer
int k=5;
v1.remove((Integer) k);//[7, 1, 5]
```

¿Qué ha pasado? el método `remove` requiere un tipo primitivo (int) para el índice. Hay que pasar los tipos adecuados en cada caso.

Autoboxing

- No se puede instanciar una clase genérica con un tipo primitivo.
- Existe una "clase envoltorio" para cada tipo primitivo:
  - Integer
  - Float
  - Double
  - Character
  - Boolean ...
- El compilador transforma automáticamente tipos primitivos en elementos de las clases envoltorio y viceversa (**autoboxing**)

```
ArrayList<Integer> v1=new ArrayList<Integer>();

v1.add(1); //[1]
int i = v1.get(0); //1
```

Ejercicio: revisa las operaciones en la clase `ArrayList` de `java.util`, analizando su comportamiento.

vector de STL

C++

- el almacenamiento se gestiona automáticamente (**array flexible**);
- acceso con índice entero (empezando en 0);
- coste constante: size, empty, push_back, pop_back;
- coste lineal: inserciones y borrados dentro del vector;

Prototipo	**Descripción**
size_type size()	dev número de elementos almacenados
bool empty()	dev `true` si está vacío
void push_back(const T& x)	añade x al final
void pop back()	elimina el último elemento
T operator[ ](size_type n)	dev el elemento en la posición n
iterator insert(iterator loc, const T& v)	inserta v antes de loc. Dev un iterador al elemento insertado
iterator begin()	dev un iterador al comienzo del vector
iterator end()	dev un iterador al final del vector
void erase(iterator p)	borra el elemento en la posición p

Uso de vector de STL

C++

Al principio del fichero se añadirá `#include <vector>`

```
vector<int> v1(3, 2); //vector de 3 enteros inicializados a 2

v1.push_back(3); //anyado al final un 3

vector<int>::iterator it;

for ( it = v1.begin(); it != v1.end(); it++) // 2 2 2 3
  cout << *it << " " ;
cout << endl;

it = v1.begin();
it++; //apunta al segundo elemento

//inserto un 7 donde apunta el iterador
v1.insert( it, 7 );                          // 2 7 2 2 3

it = v1.begin();
//borro donde apunta el iterador
v1.erase(it);                               // 7 2 2 3
```

http://www.cplusplus.com/reference/vector/vector/

Index

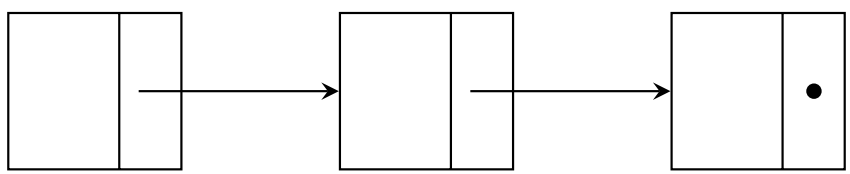
1. Objetivos
2. Librerías de Java y C++
3. Vectores
4. Listas
5. Pilas
6. Colas

Concepto de lista

Una lista es un conjunto de elementos con un orden concreto (cada elemento tiene un predecesor y un sucesor):

- puede tener una **longitud arbitraria**
- ofrece la posibilidad de insertar o eliminar un elemento en cualquier ubicación
- ofrece la posibilidad de recorrer la lista

Son colecciones de nodos que almacenan **datos y enlaces** a otros nodos.



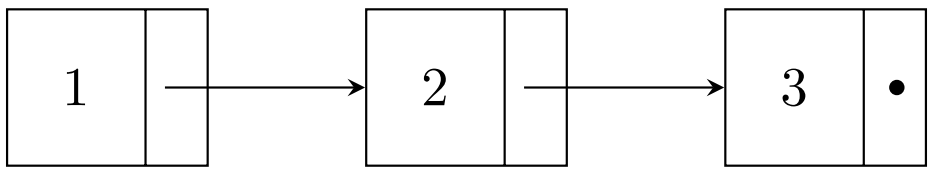
Ejemplo de implementación **(mal)** en Java

```
class Nodo {
  public int valor;
  public Nodo next;

  public Nodo(int v) {
    valor = v;
    next = null;
  }
}
```

```
Nodo n1 = new Nodo(1);
Nodo n2 = new Nodo(2);
Nodo n3 = new Nodo(3);

n1.next = n2;
n2.next = n3;
```



Esta implementación no es correcta porque nuestro objetivo es definir tipos de datos que abstraigan los detalles de implementación, p. ej:

```
Lista lista;
lista.add(1);
lista.add(2);
lista.add(3);
```

Ejemplo de implementación **(mal)** en C++

La implementación en C++ requiere el uso de punteros y memoria dinámica:

Java

```
class Nodo {
  public int valor;
  public Nodo next;

  public Nodo(int v) {
    valor = v;
    next = null;
  }
}
```

```
Nodo n1 = new Nodo(1);
Nodo n2 = new Nodo(2);
Nodo n3 = new Nodo(3);

n1.next = n2;
n2.next = n3;
```

C++

```
class Nodo {
  public:
    int valor;
    Nodo *next;

    Nodo(int v) {
      valor = v;
      next = nullptr;
    }
};
```

```
Nodo *n1 = new Nodo(1);
Nodo *n2 = new Nodo(2);
Nodo *n3 = new Nodo(3);

n1->next = n2;
n2->next = n3;
```

Ejemplo de uso **(mal)** en C++

Las funciones que recorren la lista usan el puntero **next** para pasar de un nodo al siguiente:

```
int buscar(int valor, Nodo *first) {
  Nodo *n = first;
  int pos = 0;
  while (n != nullptr) {
    if (n->valor == valor) {
      return pos;
    }
    n = n->next;
    pos++;
  }
  return -1;
}

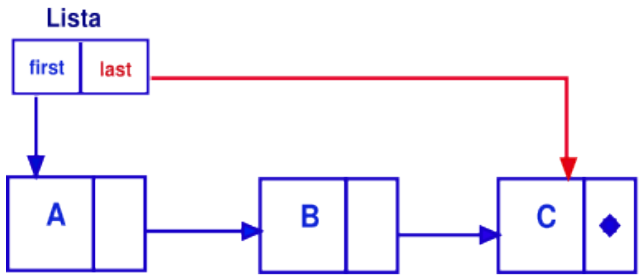
Nodo *n1 = new Nodo(1);
Nodo *n2 = new Nodo(2);
Nodo *n3 = new Nodo(3);

n1->next = n2;
n2->next = n3;

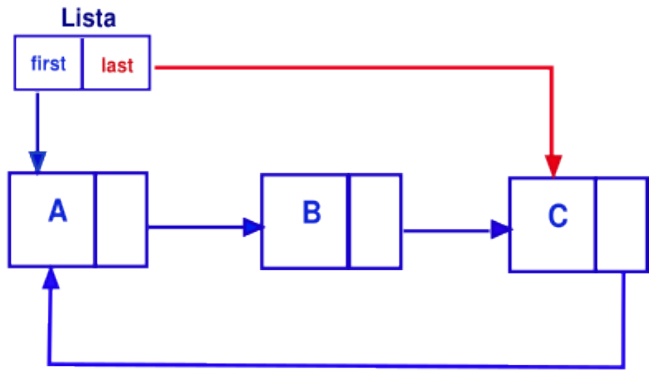
int pos = buscar(2, n1); // tenemos que pasar el primer nodo de la lista
```

Tipos de listas

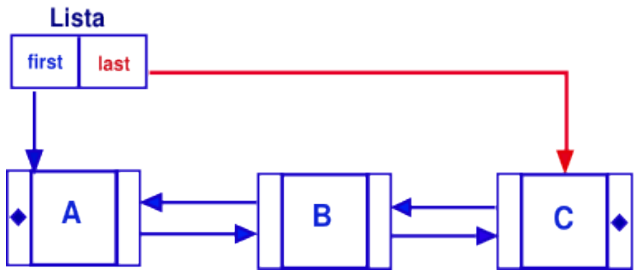
LISTA CON ENLACE SIMPLE



LISTA CIRCULAR



LISTA DOBLEMENTE ENLAZADA



Lista simple y ejemplo en C++

Es una estructura de datos formada por **nodos** (de lista) que contienen algún tipo de información y una **única referencia a otro nodo** de la lista.

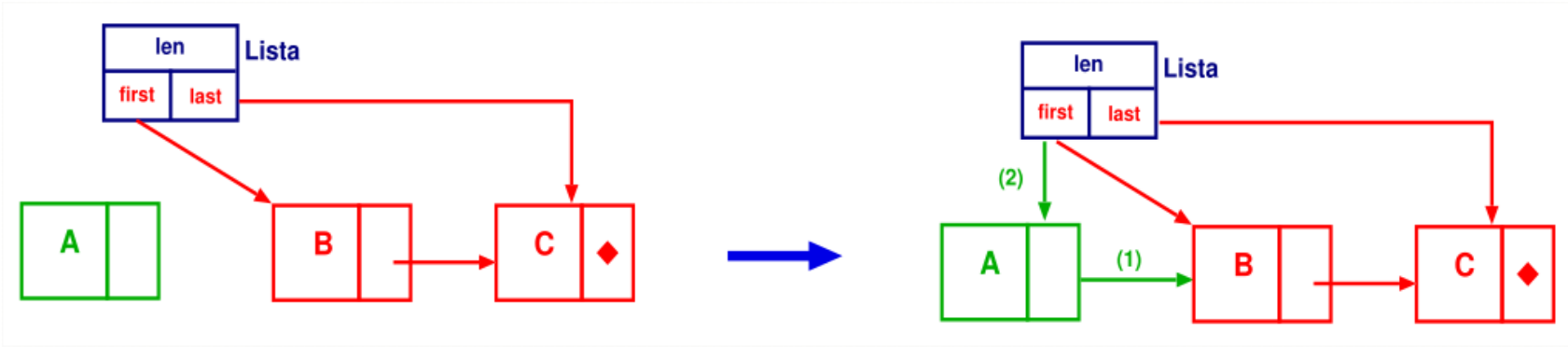
Implementación de la clase `Lista` de enteros con dos **accesos** `(first, last)` al primer y último nodo de la lista:

```
class Lista {
   private:
     class Nodo{ //nodo de la lista con enlace simple
       public:
         int valor;
         Nodo *next; //puntero al siguiente nodo

         Nodo(int v){ //constructor
           valor = v;
           next = nullptr;
         }
         ...
     };
     //variables de la lista
     Nodo *first;
     Nodo *last;
     int len;

  public:
    Lista() { first = last = nullptr; len = 0;}
    ...
```

Inserción en cabeza de lista



¿Qué versión es correcta? (La diferencia está en el `else`)

```
//INSERCION EN CABEZA EN C++
void Lista::addFirst(int v)
{
  Nodo *nuevo = Nodo(v);
  if(first == nullptr){ //si es vacia
    first = last = nuevo;
  }
  else{
    first = nuevo;
    nuevo->next = first;
  }
  len++;
}
```

```
//INSERCION EN CABEZA EN C++
void Lista::addFirst(int v)
{
  Nodo *nuevo = Nodo(v);
  if(first == nullptr){ //si es vacia
    first = last = nuevo;
  }
  else{
    nuevo->next = first;
    first = nuevo;
  }
  len++;
}
```

Búsqueda en C++

- Se compara cada elemento almacenado en la lista con el que se está buscando.
- Los métodos de comparación a utilizar se deben adecuar al tipo de datos.

```
bool Lista::find(int v)
{

  Nodo *aux;

  for (aux=first; aux!=nullptr && v!=aux->valor; aux=aux->next);

  if (aux == nullptr) return false;
  else return true;
}
```

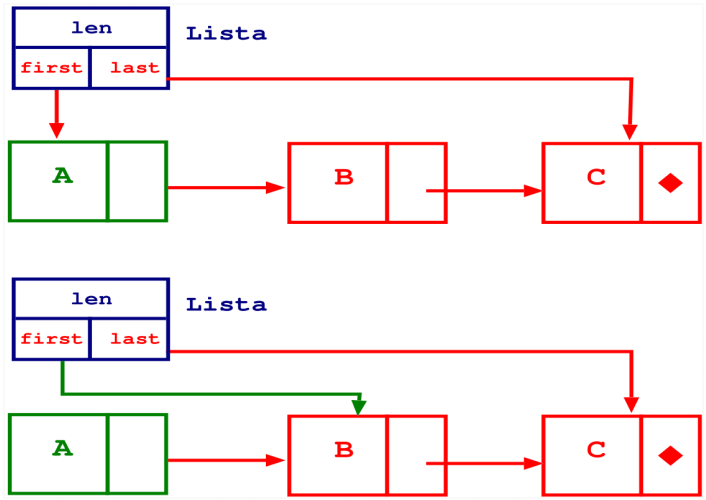
Si la lista en lugar de ser de enteros, hubiese sido de un tipo de elemento A los únicos cambios en el código anterior serían:

- sustituir el paso de parámetro por un elemento de tipo A;
- sobrecargar el operador != para la clase A;

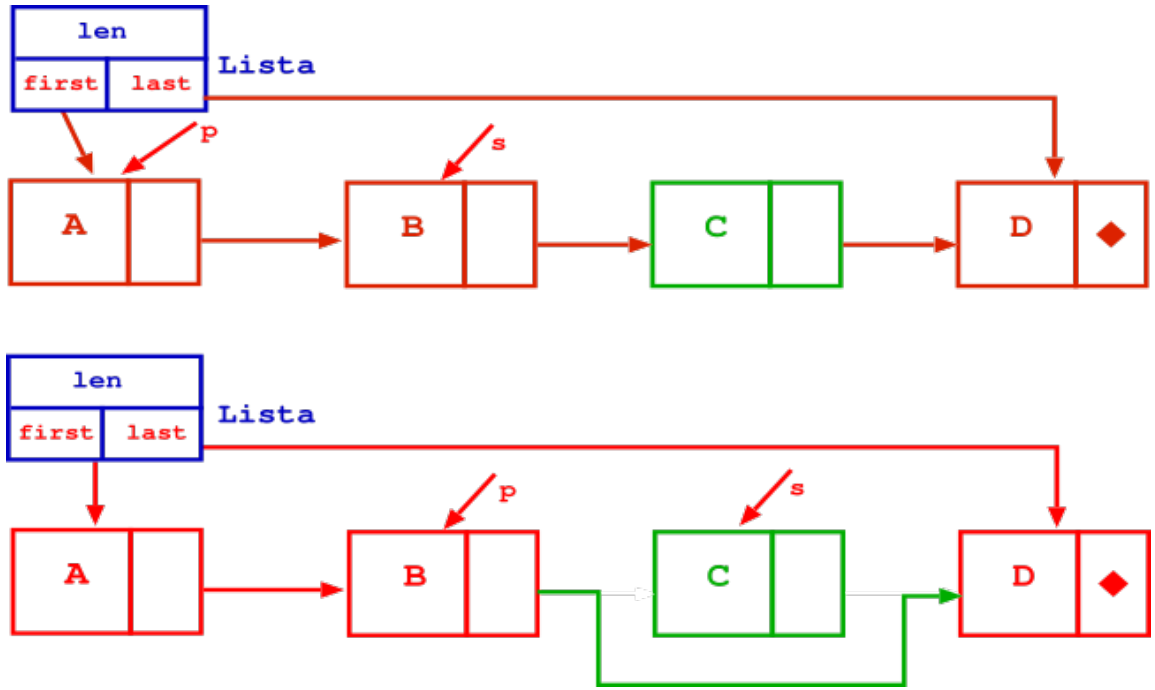
Borremos la cabeza de la lista en C++

```
int Lista::removeFirst()
{
  int devolver = -1; //valor por defecto
  Nodo *aux = first;
  if( aux != nullptr ){
    devolver = aux->valor;

    first = first->next;
    if( first == nullptr )
      last = nullptr;
    delete aux; //IMPORTANTE
    len--;
  }
  return devolver;
}
```



Borrado de un elemento de la lista



Ejercicios: calcula los costes asintóticos en el peor caso (O(-)) de las siguientes operaciones sobre una lista de caracteres

- borrar el carácter A:
- borrar el último elemento de la lista:
- añadir un elemento al final de la lista:

Uso de listas en C++

```
//main EN C++
int main()
{
    Lista lista;

    lista.removeFirst(); //no hace nada porque la lista es vacia
    lista.addFirst(20);
    lista.addFirst(10);
    lista.addFirst(40);
    lista.addFirst(30);

    if (lista.find(30))
      cout << "found" << endl;
    else
      cout << "not found" << endl;
}
```

IMPORTANTE en C++ (destructor)

Cuando un elemento es dinámico hay que liberarlo cuando sale de ámbito. Hay que definir el **destructor de la clase**.

```
Lista::~Lista(){
    Nodo *aux;
    while(first != nullptr) {
        aux = first;
        first = first->next;
        delete aux;
    }
    last = nullptr;
}
```

IMPORTANTE en C++ (constructor de copia)

Cuando pasamos por valor un argumento o lo devolvemos por valor, se invoca el **constructor de copia de la clase**.

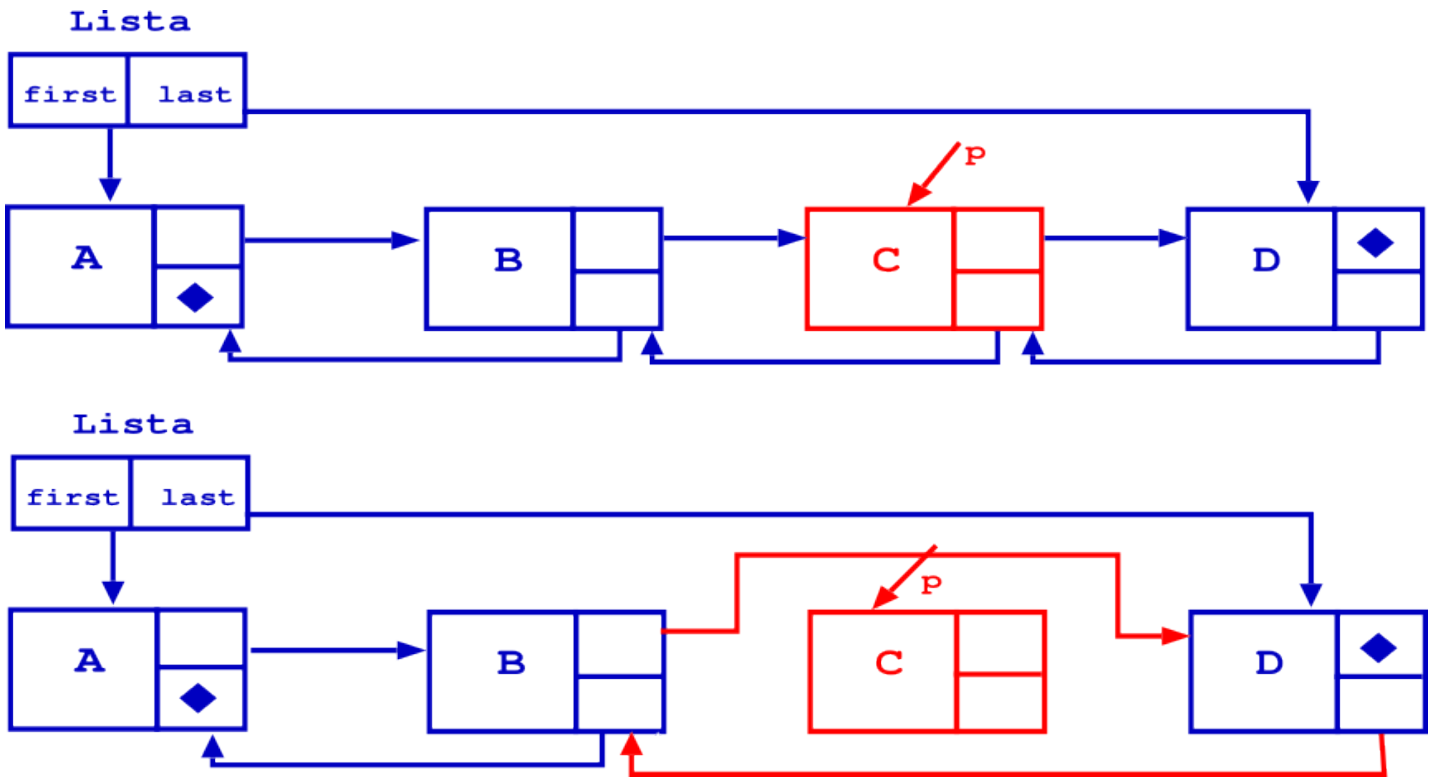
```
Lista::Lista(const Lista &p){
  Nodo *aux, *aux2;
  aux = p.first; //apunta al primer nodo de la lista p
  first = last = nullptr;
  while(aux != nullptr){
    aux2 = new Nodo(aux->valor); //reservamos memoria para el nodo
    if (first==nullptr){ //es el primero de la lista
      first = last = aux2;
    }
    else{ last->next = aux2; last = aux2; }
    aux = aux-> next; //se mueve al siguiente nodo de la lista p
  }
}
```

IMPORTANTE en C++ (operador asignación)

```
Lista & Lista::operator =(const Lista &p){
  Nodo *aux = nullptr, *aux2 = nullptr;
  if( this != &p ){
     if( first != nullptr ){ //IMPORTANTE - LIBERAR MEMORIA ANTES DE ASIGNAR NADA
       vaciarLista();
     }
    //lo mismo que en el constructor de copia
    ....
    ....
    ....
  }
  return *this;
}
```

Listas dobles

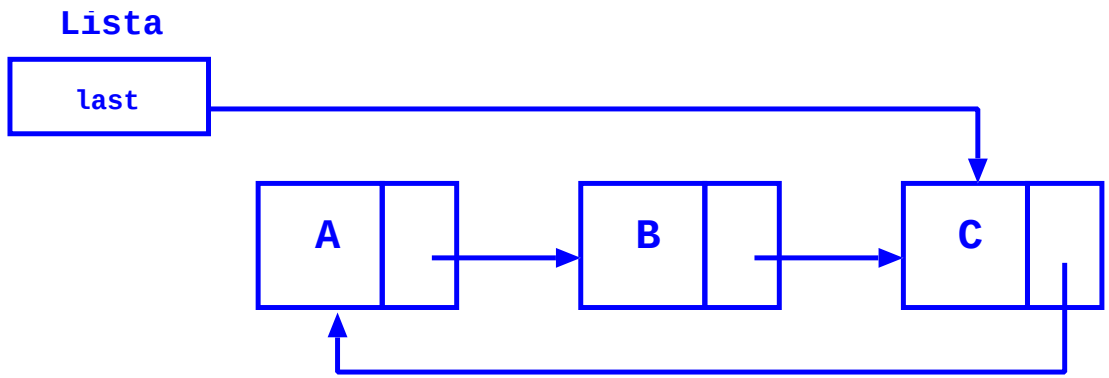
Definición de una lista en la que cada nodo de la lista tiene dos referencias



¿Qué hemos ganado respecto a las listas simples?

Listas circulares

Útiles cuando varios procesos requieren el mismo recurso durante la misma cantidad de tiempo y queremos asegurar el reparto justo del recurso.



¿En qué tiempo puede realizarse la inserción y el borrado de un elemento?

¿En qué tiempo puede realizarse la inserción y el borrado de un elemento si además la lista es doblemente enlazada?

Resumen de costes

tipo de lista	inserción cabeza	inserción cola	borrado cabeza	borrado cola	búsqueda
simple	const	const	const	lineal	lineal
doble	const	const	const	const	lineal
circular	const	const	const	lineal	lineal
circular doble	const	const	const	const	lineal
vector	lineal	const	lineal	const	lineal

En el caso de las listas contesta a las siguientes preguntas:

- ¿la tabla anterior es cierta cuando tenemos acceso sólo a la cabeza de la lista o a la cabeza y la cola?
- dependiendo de tu respuesta, ¿qué operaciones del otro caso cambian su coste?

Si tenéis alguna duda con alguno de estos costes, no tenéis más que preguntar



Resumen de costes (solución)

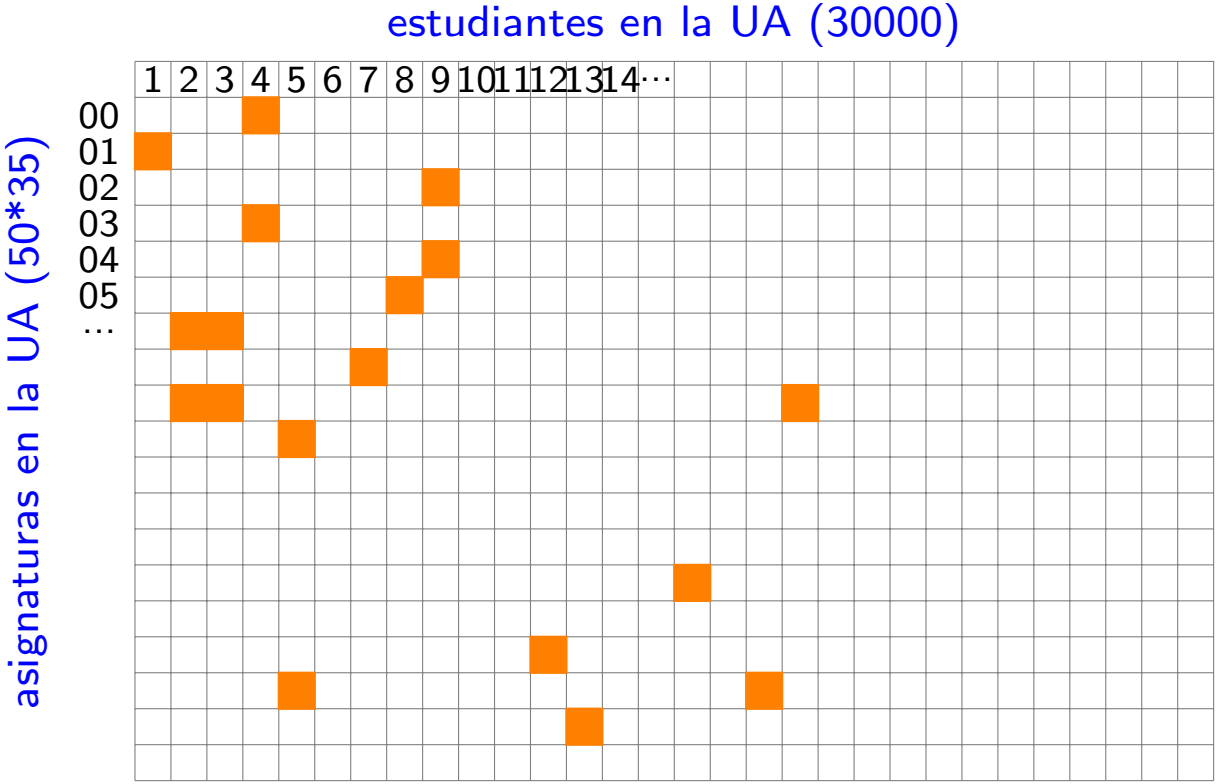
Tabla anterior: enlace a cabeza y cola de la lista.

Tabla actual: sólo tenemos acceso a la cabeza.

tipo de lista	**inserción cabeza**	**inserción cola**	**borrado cabeza**	**borrado cola**	**búsqueda**
simple	const	lineal	const	lineal	lineal
doble	const	lineal	const	lineal	lineal
circular	lineal	lineal	lineal	lineal	lineal
circular doble	const	const	const	const	lineal
vector	lineal	const	lineal	const	lineal

Uso de listas: tablas poco densas (matrices dispersas)

Son útiles cuando se descarta la opción de una tabla para almacenar información por consideraciones de espacio.



30.000*50*35=52.500.000 celdas, 300.000 celdas con información

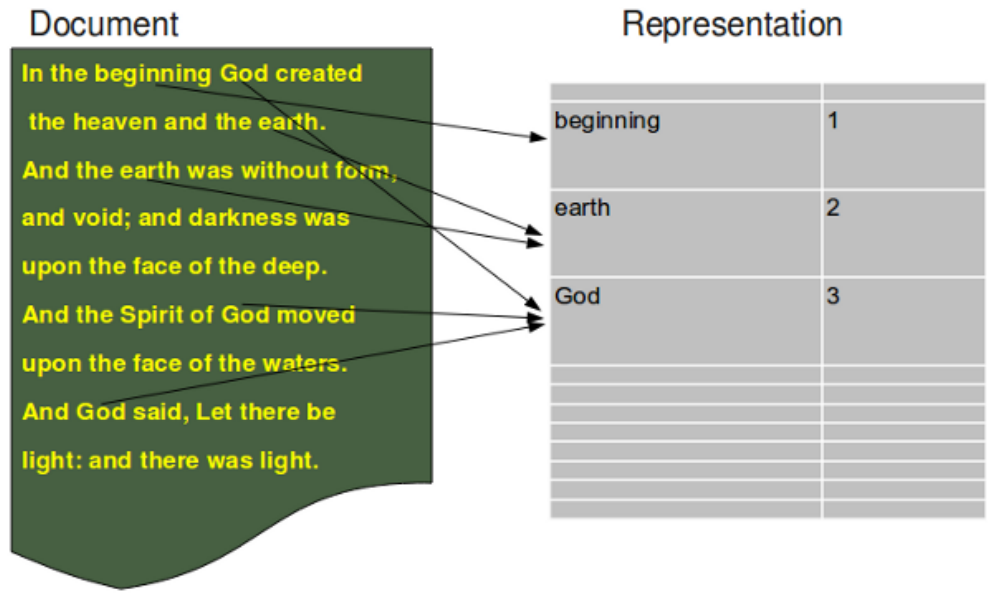
$\Rightarrow$ **0,57% ocupación**

Más ejemplos: representación de documentos

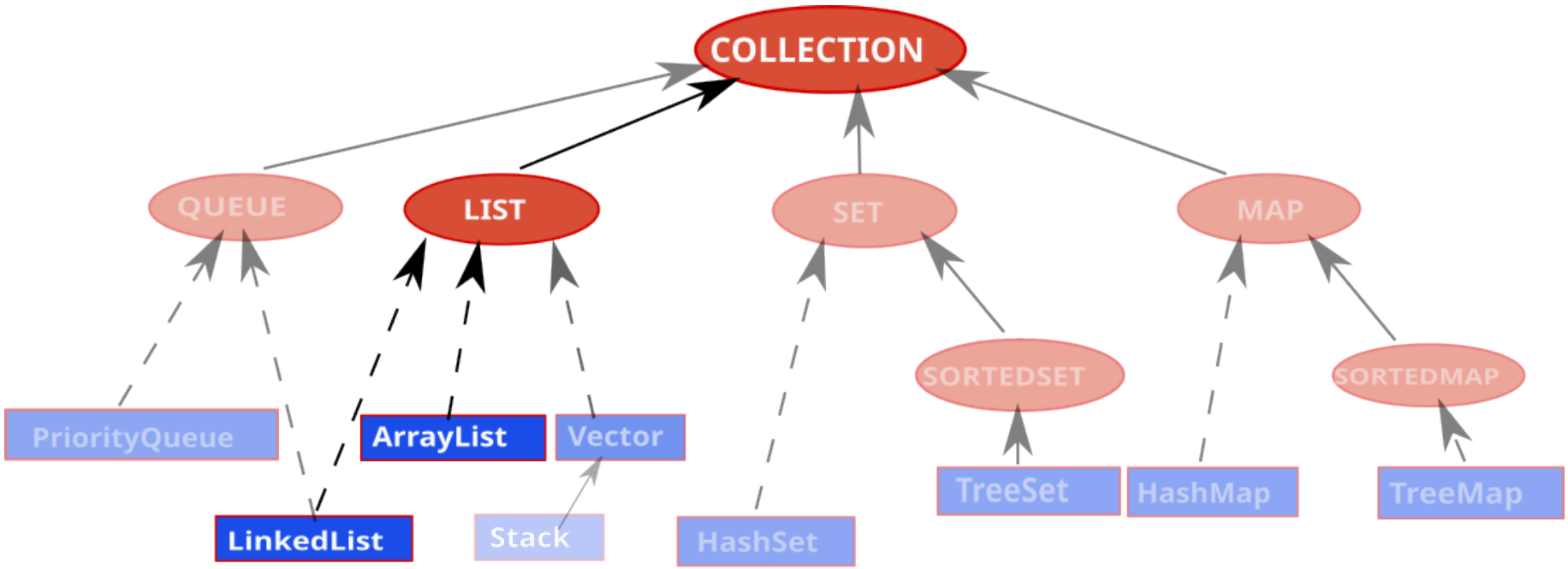
documentos

palabras del diccionario

	1	2	3	4	5	6	7	8	9	10	11	**12**	13	14	⋯
art			8												
beginning				2								1			
cat								5							
dog															
earth									2			2			
fog									1						
Good									5			3			
hot															
ice															
jam	6														
key															
leg															
man															
oil															
pub															
run															
sun															
toy															
zoo															



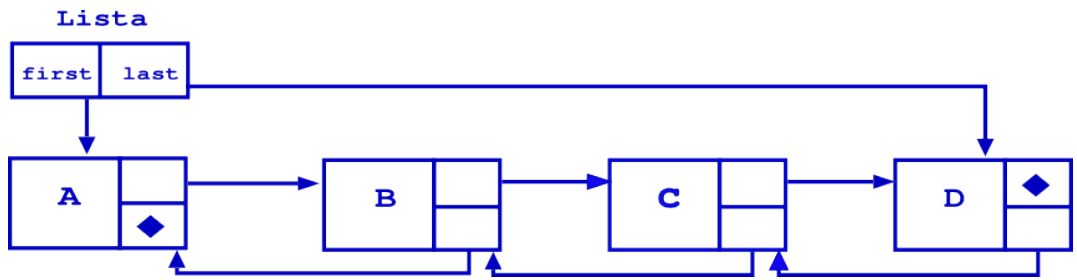
Listas en `java.util`



Listas en `java.util`: `LinkedList` y `ArrayList`

`LinkedList` es una **lista doblemente enlazada con acceso a la cabeza y cola** de la lista:

- se definen las mismas operaciones que en `ArrayList`;
- inserciones, borrados y accesos en los extremos tienen coste constante;
- resto de operaciones tienen coste lineal;



`https://docs.oracle.com/javase/7/docs/api/java/util/LinkedList.html`

`https://docs.oracle.com/javase/7/docs/api/java/util/ArrayList.html`

LinkedList y ArrayList

Ventajas y desventajas de su uso:

`LinkedList` Una implementación de una lista doblemente enlazada. La **modificación es poco costosa** para cualquier tamaño, pero el **acceso aleatorio es lento**. Útil para implementar colas, colas dobles y pilas.

- `getFirst`, `getLast`, `removeFirst`, `removeLast`, `addFirst`, `addLast`

`ArrayList` Una lista implementada utilizando un array de dimensión modificable. Es **costoso añadir o borrar un elemento** cerca del principio de la lista si ésta es grande, pero es relativamente poco costoso de crear y **rápido para acceso aleatorio**.

`list` de STL

C++

La clase list (lista) es una lista doblemente enlazada:

- la navegación a través de la lista se realiza de forma similar al `vector`;
- se proporcionan iteradores, que son bidireccionales;
- eficiencia similar a las listas de java.util;
- también tiene las funciones miembro `empty`, `size`, `clear` y operaciones canónicas;

Prototipo	**Descripción**
`void push_back(const T& e)`	añade elemento al final
`void push_front(const T& e)`	añade elemento al principio
`T& front()`	dev referencia al primer elemento
`T& back()`	dev referencia al último elemento
`T& pop_front()`	borra el primer elemento y lo devuelve
`void remove(const T& e)`	borra todos los elementos iguales a `e`*
`void sort()`	ordena la lista de forma ascendente*

*Los operadores $==$ y $<$ deben estar definidos para el tipo de elemento almacenado en la lista.

Uso de iteradores con `list` de STL

C++

Prototipo	Descripción
`iterator begin()`	iterador al primer elemento
`iterator end()`	iterador al final
`iterator insert(iterator pos, const T& e)`	inserta e en pos; dev iterador al insertado
`iterator erase(iterator pos)`	borra el elemento en pos; devuelve un iterador al siguiente

`http://www.cplusplus.com/reference/list/list/`

Ejemplo de uso de iteradores con listas en C++

```
#include <iostream>
#include <list>
int main(){
  list<int> lis;

  lis.push_back(20); lis.push_back(10);
  lis.push_back(40); lis.push_back(30);

  list<int>::iterator i; //iterador a la lista
  for (i = lis.begin(); i != lis.end(); i++)                    //20 10 40 30
    cout << *i << " ";
  cout << endl;

  for (int j = 5; j < 15; j+=5) //insertamos 2 elementos con iterador
    lis.insert(lis.end(), j);
  for (i = lis.begin(); i != lis.end(); i++)                  //20 10 40 30 5 10
     cout << *i << " ";
  cout << endl;

  list<int>::iterator j;
  i = lis.begin();
  i++; i++;
  j = lis.erase( i ); //borramos elemento con iterador

  for (i = lis.begin(); i != lis.end(); i++)                    //20 10 30 5 10
    cout << *i << " ";
  cout << endl;
  cout << *j << endl; //elemento apuntado por el iterador devuelto en erase
}
```

Index

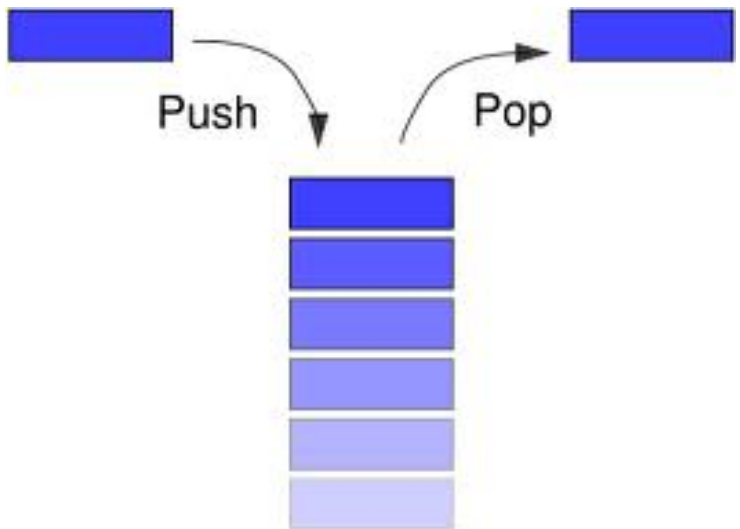
1. Objetivos
2. Librerías de Java y C++
3. Vectores
4. Listas
5. Pilas
6. Colas

4. Concepto de pila

Una pila es una estructura de datos lineal en la cual el acceso está limitado al último elemento insertado (LIFO) $\rightarrow$ acceso por un único extremo.

[**L**ast **I**n **F**irst **O**ut]

Las operaciones que caracterizan al tipo de datos pila son: apilar, desapilar, cima, esvacia.

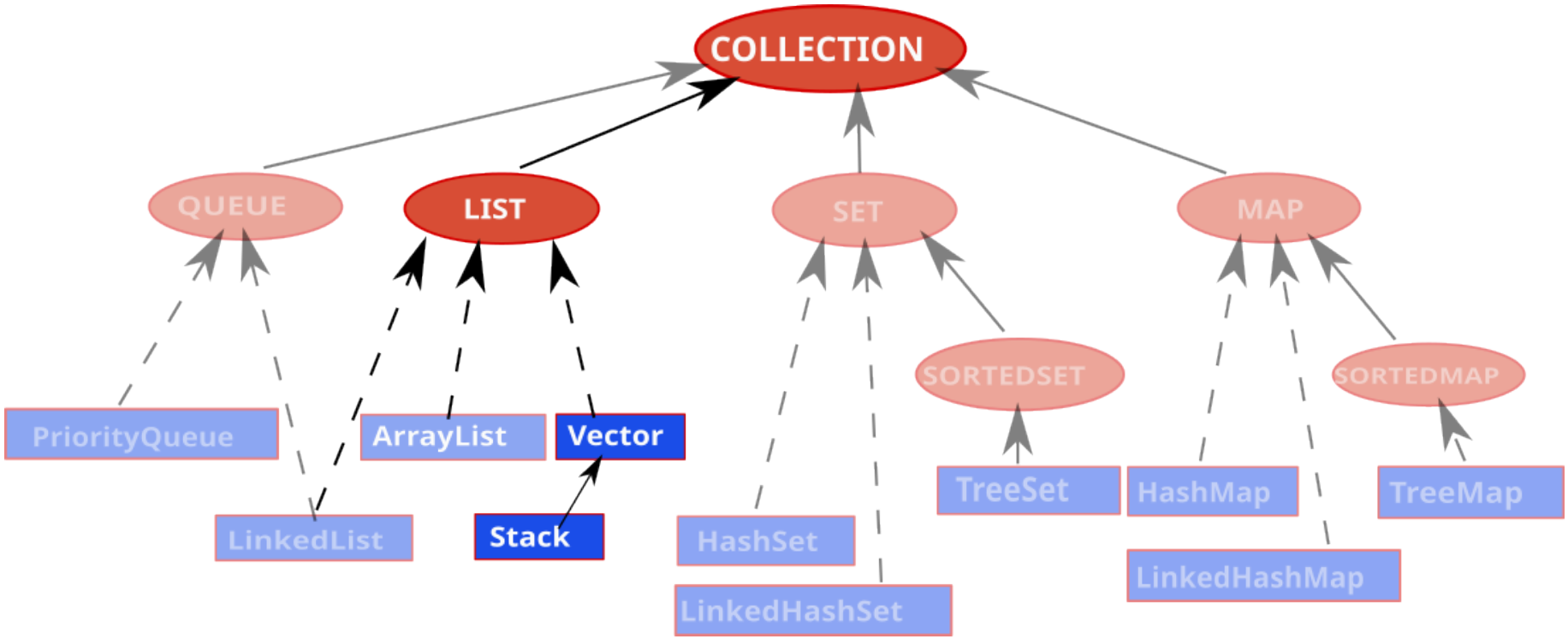


Con implementaciones adecuadas el coste de todas las operaciones es $\mathrm{O}(1)$:

- con listas enlazadas sólo insertamos, borramos y buscamos en la cabeza(principio) o cola(final) de la lista;
- con arrays sólo insertamos, borramos y buscamos al final del array;

Pilas en `java.util` y STL

`java.util`



STL

Por defecto el contenedor de stack en STL es un deque (pero puede usarse `vector` o `list`)

Pilas en `java.util` y STL

Los métodos definidos en la clase `Stack` de `java.util` y `stack` de STL son:

Java	**C++**	**Descripción**
bool empty()	bool empty()	devuelve true si pila vacía
E peek()	E& top()	devuelve referencia a la cima de la pila
E pop()	void pop()	borra el contenido de la cima
E push(E v)	void push (const E& v)	inserta v en la cima

En Java:

```
Stack<String> pila = new Stack<String>();
```

En C++:

```
stack<string> pila;
```

Pilas en STL: ejemplo de uso

```
//CODIGO EN C++
stack<int> p;

p.push(1);
p.push(2);
p.push(3);
p.push(4);
cout << "cima = " << p.top() << endl; //cima = 4

p.pop();
cout << "cima = " << p.top() << endl; //cima = 3
```

Desgraciadamente en STL no podemos hacer

```
cout << p << endl; //ERROR DE COMPILACION
```

Pila: algunas aplicaciones

Herramientas para el diseño de compiladores:

- programa que comprueba la parentización, $\{\}[]$ [2]
  - cada vez que aparece un "(", se introduce en la pila
  - cada vez que aparece un ")", desapilamos
  - la cadena de paréntesis es correcta si al finalizar el recorrido de la cadena, la pila está vacía
  - CORRECTO: (()), ()(), INCORRECTO:)(,)((, ())(

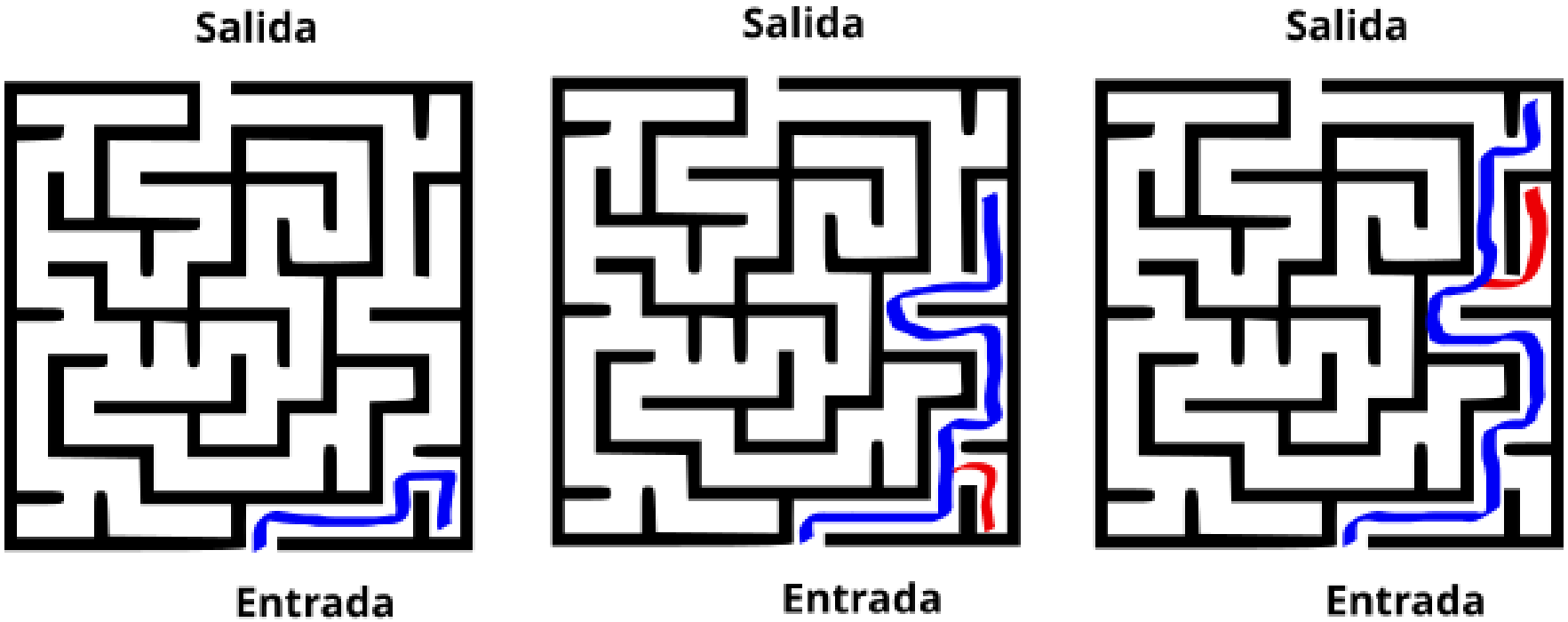
- comprobación de código en lenguajes de marcado HTML/XHTML
  - CORRECTO: $< b >< i >$ Hola $< /i >< /b >$
  - INCORRECTO: $< b >< i >$ Hola $< /b >< /i >$

- evaluación de expresiones aritméticas en lenguajes de programación

[2] ojo, habría que comprobar que el que se desapila es del mismo tipo que el que tenemos, y en el orden adecuado ...

Aplicación: salida de un laberinto

Se usa una **pila** como estructura de datos auxiliar para resolver el problema de encontrar la salida de un laberinto. Debe elegirse un camino, y si no lleva a la salida hay que **volver atrás** y continuar en una dirección diferente, así hasta encontrar la única salida existente.



Index

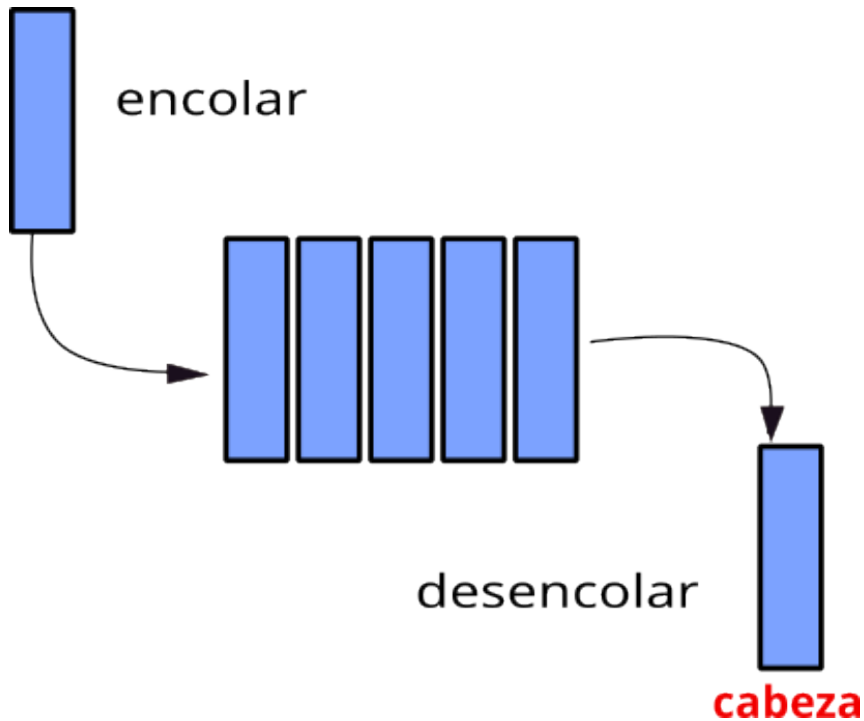
1. Objetivos
2. Librerías de Java y C++
3. Vectores
4. Listas
5. Pilas
6. Colas

Concepto de cola

Una cola es una estructura de datos lineal en la cual el acceso está limitado al elemento menos reciente (FIFO) $\rightarrow$ inserción por un extremo y borrado por otro.

[**F**irst **I**n **F**irst **O**ut]

Las operaciones que caracterizan al tipo de datos cola son: encolar, desencolar, cabeza, esvacía.



Con implementaciones adecuadas el coste de las operaciones es $\mathrm{O}(1)$

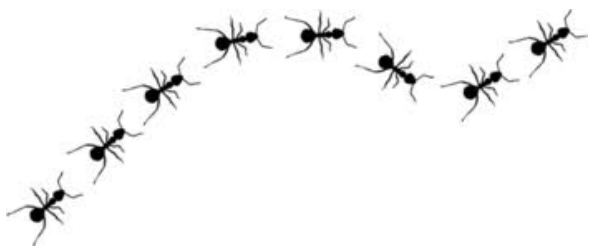
Cola: ejemplos de uso

En la vida real:

- cola en la parada de un autobús
- cola en la entrada a un espectáculo
- cola de aviones para despegar
- cola de impresión en una impresora ...







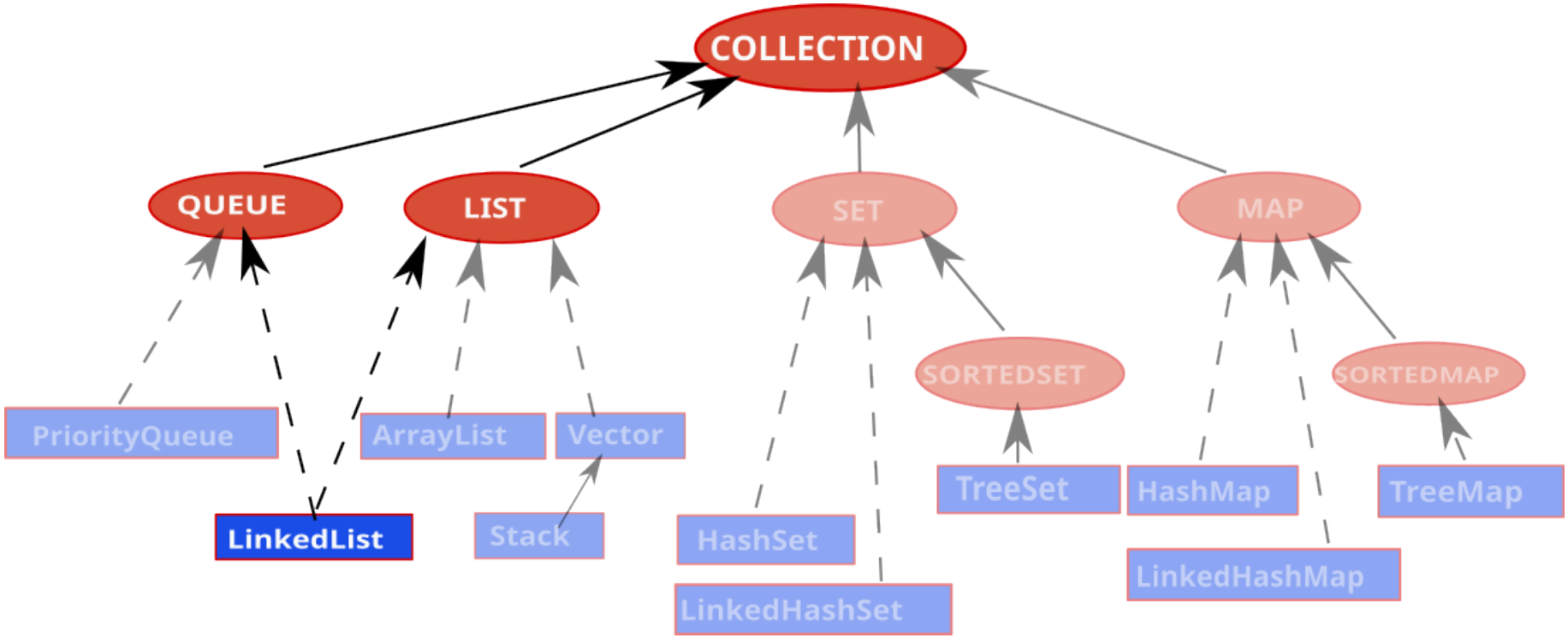


En algoritmia:

- colas para recorridos en árboles y grafos

Colas en `java.util` y STL

`java.util`



En `java.util` sólo se ha definido un interfaz

STL

El contenedor es queue

Colas en `java.util` y STL

Los métodos definidos en el interface `Queue` de `java.util` y `queue` de `STL` son:

Java	**C++**	**Descripción**
bool isEmpty()	bool empty()	devuelve true si pila vacía
E peek()	E& front()	devuelve referencia a la cabeza de la cola
E poll()	void pop()	borra el contenido de la cabeza
boolean offer(E v)	void push (const E& v)	inserta v al final

En Java:

```
Queue<String> qe = new LinkedList<String>();
```

En C++:

```
queue<string> qe;
```

Colas en STL: ejemplo de uso

```
//CODIGO EN C++
queue<string> mq;
mq.push("ele1"); mq.push("ele2"); mq.push("ele3"); //insertamos

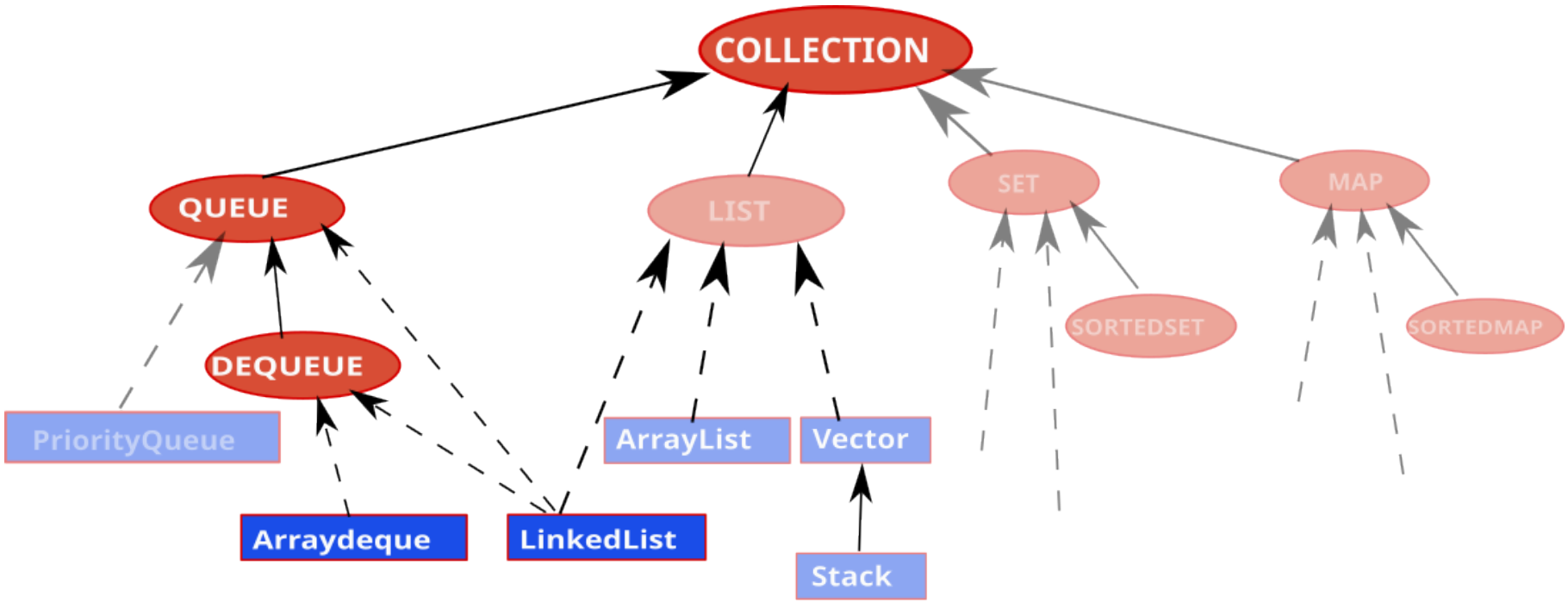
//OPERACIONES CON COLA
cout << "Front = " << mq.front() << endl; //Veo la cabeza: ele1
mq.pop();                                 //Elimino la cabeza: ele1
cout << "Front = " << mq.front() << endl; //Veo la cabeza: ele2

//VISUALIZAR LA COLA EN C++ ... solo se puede hacer desencolando
```

Doble cola (bicola)

Permiten inserciones y borrados en ambos extremos.

`java.util`



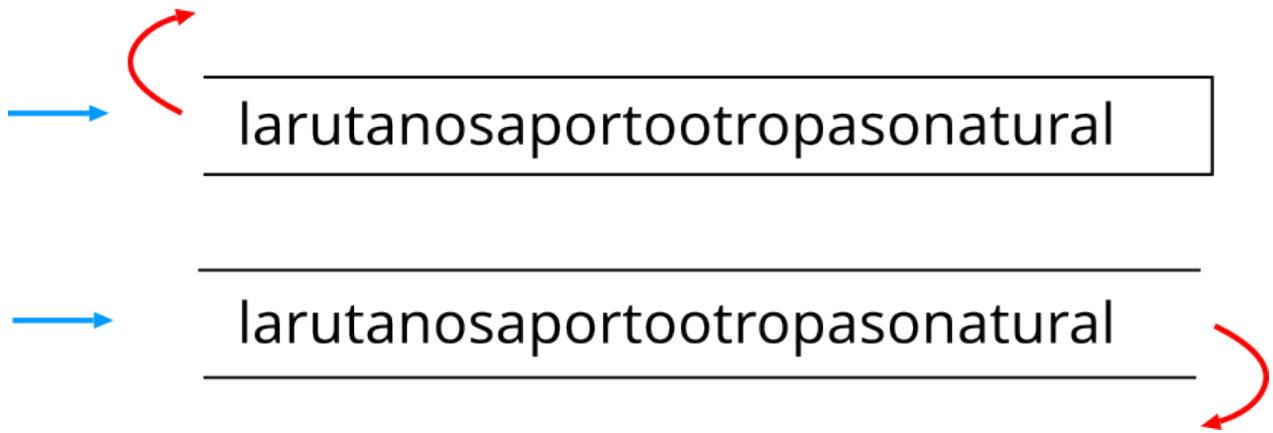
`STL`

En `STL` tenemos `deque`, con una funcionalidad similar al `vector` pero con inserción y borrado eficiente en el principio. Puede usar iteradores.

Aplicación de pilas y colas: búsqueda de palíndromos

Con una pila y una cola ...

- se introducen las letras de una frase en ambas estructuras
- se extrae una letra de cada estructura
- si cada pareja de letras extraídas coincide, la frase es palíndroma



El mismo algoritmo se puede aplicar utilizando una bicola extrayendo elementos de la cabeza y la cola a la vez.